Brújula Viajes

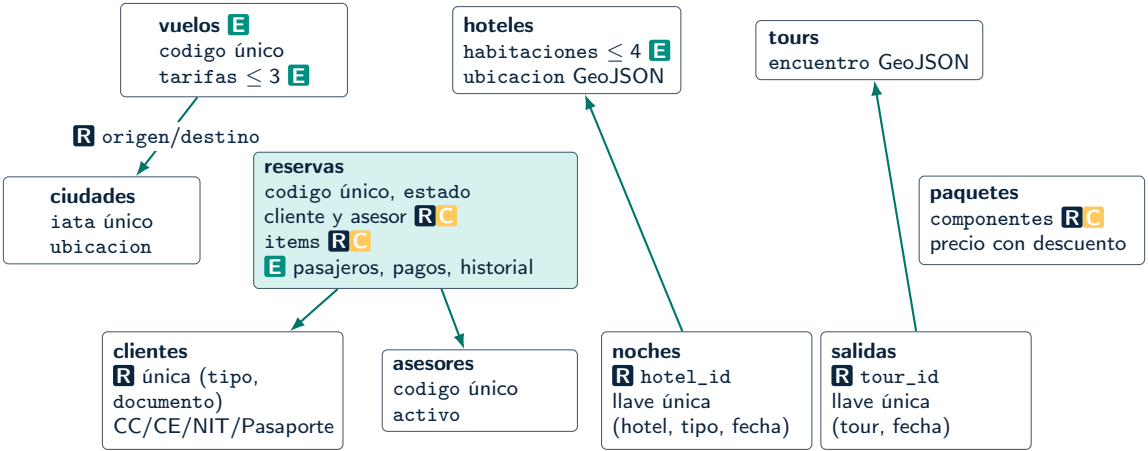
Modelado de documentos, API y frontend

Antonio Gutiérrez, David Rodríguez,
David Pascagaza y Sara Torres

Ingeniería de Datos, Universidad del Rosario

10 de noviembre de 2026

Modelo de documentos: 10 colecciones



E embebido R referencia C copia congelada al vender · los ítems de la reserva apuntan a vuelos, hoteles, tours o paquetes (R C)

Estructura de las colecciones

vuelos

codigo, aerolinea: texto
origen, destino: texto IATA
sale, llega: fecha
E tarifas[]: { nombre: texto, precio: número, equipaje: texto, sillas, libres: enteros }

ciudades

iata, nombre: texto
ubicacion: punto GeoJSON

tours

nombre, descripcion: texto
ciudad: IATA
horas: número
incluye[]: texto
E encuentro: { lugar: texto, ubicacion: GeoJSON }

hoteles

nombre, direccion: texto ciudad: IATA
estrellas: entero servicios[]: texto
ubicacion: punto GeoJSON
E habitaciones[]: { tipo: texto, capacidad, cantidad: enteros, precio_noche: número }

noches

R hotel_id → hoteles
tipo: texto
fecha: fecha
libres: entero

salidas

R tour_id → tours
fecha: fecha
precio: número
cupos, libres: enteros

E embebido R referencia C copia [] arreglo _id: ObjectId que genera MongoDB

Estructura de las colecciones

clientes

tipo, documento, nombre, correo, telefono: texto
E preferencias: { favoritos[]: IATA, presupuesto: número, con_ninos: bool }

asesores

codigo, nombre, correo: texto
activo: bool

paquetes

nombre: texto destino: IATA descuento: número
desde, hasta: fecha
E componentes[]: { tipo: texto, R ref_id: ObjectId, C detalle: objeto, nombre: texto, precio: número }

reservas

codigo, estado: texto creada: fecha
R cliente_id → clientes C cliente: { tipo, documento, nombre: texto }
R asesor_id → asesores C asesor: texto
E items[]: { tipo: texto, ref_id: ObjectId, detalle: objeto, nombre: texto, ciudad: IATA, inicio, fin: fecha, cantidad: entero, precio, subtotal: número }
E pasajeros[]: { nombre, documento: texto, nacimiento: fecha }
total, pagado: número
E pagos[]: { fecha: fecha, valor: número, medio: texto }
E historial[]: { antes, despues: texto, fecha: fecha, usuario: texto }

E embebido R referencia C copia [] arreglo _id: ObjectId que genera MongoDB

Decisiones de modelado y su justificación

1 ¿Embebido o colección aparte?

Tarifas (≤ 3) y habitaciones (≤ 4): **embebidas**; se descuentan con su producto. Salidas de tour: **aparte**; crecen sin límite (única por tour+fecha).

2 Habitaciones por noche

Colección `noches`: un documento por (hotel, tipo, fecha), llave única y fecha a 00:00 UTC. Generación **idempotente**: regenerar no pisa lo vendido.

3 Reserva: copias y referencias

Copia de cliente, precios y subtotales: la reserva no cambia si el catálogo sube. **Referencias** mínimas para devolver el inventario al cancelar.

4 Paquetes: REF + COPIA

Componente = `ref_id` + copia de nombre y precio; el total sale de las copias con el descuento. **Inmutables**: la cancelación reconstruye el inventario desde ellos.

5 Pagos e historial: embebidos

Se leen con la reserva y crecen poco. Un pago = **una escritura atómica**: `$push` + `$inc` + estado + evento en el historial.

6 Arreglos sin límite

Las reservas NO van dentro del cliente. Cotas en los validadores `$jsonSchema`: tarifas 3, habitaciones 4, ítems y pasajeros 20 por reserva.

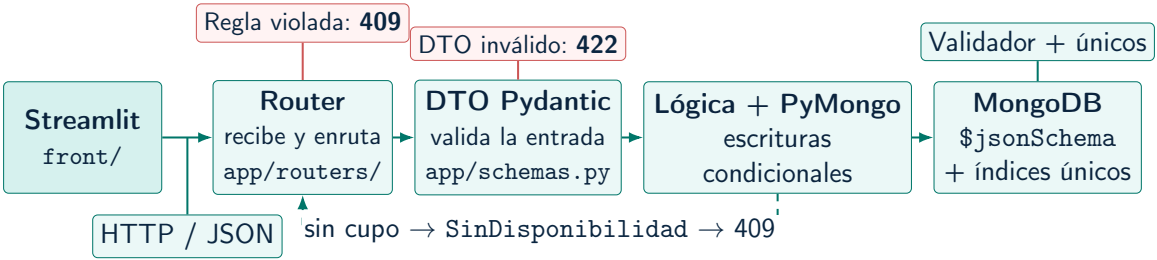
Ejemplo: un documento de `reservas`

```
{ "codigo": "RV-2026...",
  "estado": "Pagada",
  "cliente": { "tipo": "CC",
    "documento": "1234567890",
    "nombre": "Ana Cliente" },        <- copia
  "asesor": { "nombre": "J. Asesor" },
  "items": [ { "tipo": "vuelo",
    "ref_id": "...",                  <- referencia
    "detalle": { "tarifa": "Clasica" },
    "nombre": "Vuelo BV1234",
    "precio": 250000, "cantidad": 2,
    "subtotal": 500000 } ],           <- copia
  "pasajeros": [ ... ],               <- embebidos
  "pagos": [ { "valor": 250000, "medio": "PSE" },
             { "valor": 250000, "medio": "Tarjeta" } ],
  "historial": [ { "antes": "Pendiente",
                   "despues": "Pagada" } ],
  "total": 500000, "pagado": 500000 }
```

Qué se vende con este diseño

- **Precio congelado:** si el catálogo cambia, la reserva ya vendida no se mueve.
- **Cancelación exacta:** `detalle` guarda lo mínimo para devolver sillas, noches y cupos.
- **Consultas sin $lookup:** filtrar por cliente usa la copia del documento.
- Estructura real del esquema; valores ilustrativos.

Del clic en Streamlit a MongoDB



El frontend **nunca** se conecta directo a la base: toda operación pasa por la API. FastAPI valida el DTO **antes** de la lógica del handler: el rechazo del DTO ocurre antes de cualquier escritura. El descuento de inventario es **una escritura atómica por documento** (filtro `libres >= n` + `$inc`): dos clientes no venden la misma última silla.[a]

[a] Atomicidad por documento: https://www.mongodb.com/docs/manual/core/write-operations-atomicity/

Protecciones: qué responde la API y por qué

Códigos de respuesta

200/201/204	éxito
404	el recurso no existe
409	**sin cupo** o llave única duplicada
422	**DTO inválido** (Pydantic) o documento fuera del `$jsonSchema`
500	error interno, sin detalles al cliente
503	MongoDB no responde

Errores saneados

Los 500 registran el error completo en el log del servidor, pero al cliente solo llega un mensaje genérico: nada de trazas ni cadenas de conexión.

Candado del esquema

Los validadores e índices solo se (re)aplican cuando `BRUJULA_APLICAR_ESQUEMA` vale `'si'`. El candado controla cuándo se aplica el esquema; no bloquea las escrituras de la API.

Arquitectura y decisiones de implementación

Backend (FastAPI)

- Un router por recurso; los 18 endpoints mínimos cubiertos (cancelación y 3 reportes incluidos).
- Reportes B1–B3: **pipelines de agregación**.
- Multi-ítem sin transacciones: cotiza, descuenta y **compensa** si algo falla.

Frontend (Streamlit)

- Inicio con login: correo del asesor + su código.
- 4 módulos: Buscar y reservar, Reservas, Catálogo y clientes, Reportes.
- Cliente con identidad compuesta: tipo + número de documento.
- Todo por la API; el mapa usa las coordenadas de `ciudades`.

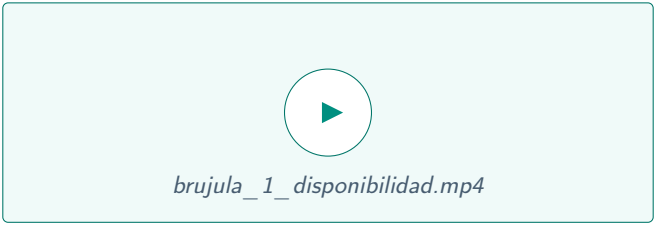
Datos demo reproducibles

- `datos_mongo.py` carga **por la API** y reparte las reservas en 3 meses.
- 6 ciudades, 12 vuelos, 3 paquetes, 17 clientes, 3 asesores y **22 reservas**.
- Solo toca la base `brujula_viajes`.

Video 1: disponibilidad y reserva sin sobreventa

1 Buscar y reservar

Búsqueda de vuelos, tarifas y sillas libres · una tarifa agotada se muestra sin cupo · al reservar, la disponibilidad desciende.



brujula_1_disponibilidad.mp4

Video 1 en línea

Video 2: registrar un paquete

2 Catálogo y clientes: paquetes

Registro de un paquete: nombre, destino, descuento y ventana de venta · componentes de vuelo, hotel o tour con precio por persona · el sistema calcula el precio final y el paquete aparece en el catálogo.



brujula_2_paquetes.mp4

Video 2 en línea

Video 3: reportes BI

3 Reportes

B1 ventas por mes (pesos y número de reservas) · **B2** destinos más vendidos en el mapa (tamaño = ingresos, color = pasajeros) · **B3** ocupación de vuelos por aerolínea.



brujula_3_reportes.mp4

Video 3 en línea

Cierre



Un modelo de documentos que **vende sin sobreventa**, una API que **protege las reglas** en cada capa y un frontend que lo hace **usable** para el asesor.

Tu rumbo a recuerdos inolvidables.